



Annexe 2 — Java RMI avec Spring Boot

Année 2026–2027 Enseignant Bachar Rima

À QUOI SERT CETTE ANNEXE

Elle accompagne la **partie B bonus du TP 1**. Elle donne ce qu'il faut pour faire tourner une application Java RMI à l'intérieur d'applications Spring Boot : structure de projets, dépendances exactes, versions épinglées, et procédure d'exécution.

Elle ne remplace pas l'annexe « Spring Boot Crash Course » (2026–2027), qui explique les *notions* : inversion de contrôle, injection de dépendances, beans, `ApplicationContext`, cycle de vie d'une application non web. Lisez-la d'abord si ces mots ne vous disent rien encore.

1. Versions — épinglées et testées

ÉLÉMENT	VERSION	REMARQUE
JDK	17 (minimum)	validé sur 17.0.10 et 21.0.5
Spring Boot	3.5.16	version épinglée dans le <code>pom.xml</code> racine
Maven	3.6.3 minimum	exigence de Spring Boot 3.5 ; validé avec 3.9.6
Dépendance	<code>spring-boot-starter</code>	et rien d'autre

ATTENTION

N'écrivez jamais « dernière version » dans un projet destiné à être reproduit par quelqu'un d'autre. Épinglez une version précise, et **dites laquelle vous avez testée** dans votre README : c'est exactement ce que le barème appelle la reproductibilité.

PAS DE WEB, PAS DE REST

Spring Boot n'implique pas une application web. N'ajoutez **ni** `spring-boot-starter-web`, **ni** contrôleur REST, **ni** Tomcat : ce serait du poids inutile, et REST sera étudié en CM 04 puis pratiqué au TP 3.

2. Structure : un projet Maven, trois modules

Un seul projet Maven, **trois modules**, **quatre fichiers** `pom.xml`, chacun dans son dossier :

```
vet-rmi-boot/      racine du projet : c'est ICI que l'on lance mvn
|-- pom.xml        POM parent et agregateur
|-- vet-common/
|   |-- pom.xml    module Java simple : AUCUNE dependance
|   |-- src/main/java/... interfaces distantes, types serialisables
|-- vet-server/
|   |-- pom.xml    application Spring Boot : le serveur
|   |-- src/main/... servant, publication, application.properties
|-- vet-client/
|   |-- pom.xml    application Spring Boot : le client
|   |-- src/main/... lookup, logique client et CLI de l'etape A8
```

FICHIER	RÔLE	DÉPEND DE
vet-rmi-boot/pom.xml	parent et agrégateur (<code>packaging pom</code>)	—
vet-rmi-boot/vet-common/pom.xml	contrats partagés, Java seul	rien
vet-rmi-boot/vet-server/pom.xml	serveur Spring Boot, JAR exécutable	vet-common, spring-boot-starter
vet-rmi-boot/vet-client/pom.xml	client Spring Boot, JAR exécutable	vet-common, spring-boot-starter

C'est le découpage `common` / `server` / `client` de la partie A, devenu un projet Maven multi-modules.

2.1. Le POM racine : vet-rmi-boot/pom.xml

Listing 1 – Fichier : vet-rmi-boot/pom.xml

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0">
3    <modelVersion>4.0.0</modelVersion>
4
5    <!-- Herite de Spring Boot : gestion des versions et des plugins -->
6    <parent>
7      <groupId>org.springframework.boot</groupId>
8      <artifactId>spring-boot-starter-parent</artifactId>
9      <version>3.5.16</version>          <!-- version EPINGLEE -->
10     <relativePath/>
11   </parent>
12
13   <groupId>fr.umontpellier.hai704i</groupId>
14   <artifactId>vet-rmi-boot</artifactId>
15   <version>1.0.0</version>
16   <packaging>pom</packaging>          <!-- parent et agregateur -->
17
18   <properties>
19     <java.version>17</java.version>
20     <project.build.sourceEncoding>UTF-8</project.build.sourceEncoding>
21   </properties>
22
23   <!-- Les trois modules construits par "mvn clean install" -->
24   <modules>
25     <module>vet-common</module>
26     <module>vet-server</module>
27     <module>vet-client</module>
28   </modules>
29 </project>

```

Le parent `spring-boot-starter-parent` fixe la version de Spring Boot et, avec elle, celle des dépendances qu'il gère : c'est la seule version de Spring à écrire. `packaging pom` : ce projet ne produit pas de JAR, il regroupe les modules.

2.2. Hériter et dépendre : deux relations distinctes

```
spring-boot-starter-parent 3.5.16
| herite (<parent>)
v
vet-rmi-boot/pom.xml          parent + agregateur (<modules>)
| herite (<parent>)
+-- vet-common/pom.xml
+-- vet-server/pom.xml --- depend de (<dependency>) ---> vet-common
+-- vet-client/pom.xml --- depend de (<dependency>) ---> vet-common
```

- `<modules>`, dans le POM racine, dit à Maven **quoi construire** : `mvn clean install` lancé dans `vet-rmi-boot/` construit les trois modules, `vet-common` en premier puisque les deux autres en dépendent.
- `<parent>`, dans chaque module, dit **de quoi il hérite** : `groupId`, `version`, Java 17 et les versions gérées par Spring Boot. Hériter n'ajoute **aucune** dépendance.
- `<dependency>` vers `vet-common` est une relation **distincte** : c'est elle qui place les contrats sur le *classpath* du serveur et du client.
- `vet-common` ne dépend ni de Spring, ni du serveur, ni du client ; le client ne dépend pas du serveur. C'est la règle de la partie A, désormais vérifiée par Maven.

2.3. Le module commun : vet-rmi-boot/vet-common/pom.xml

Listing 2 – Fichier : vet-rmi-boot/vet-common/pom.xml

```
1 <?xml version="1.0" encoding="UTF-8"?>
2 <project xmlns="http://maven.apache.org/POM/4.0.0">
3   <modelVersion>4.0.0</modelVersion>
4
5   <parent>
6     <groupId>fr.umontpellier.hai704i</groupId>
7     <artifactId>vet-rmi-boot</artifactId>
8     <version>1.0.0</version>
9   </parent>
10
11   <artifactId>vet-common</artifactId>      <!-- groupId et version herites -->
12
13   <!-- AUCUNE dependance : ni Spring, ni vet-server, ni vet-client.
14        Interfaces distantes et types transmis par valeur, en Java seul. -->
15 </project>
```

Seul l'`artifactId` est écrit : `groupId` et `version` viennent du parent. `vet-common` n'a **aucune dépendance** : aucun JAR Spring n'est sur son *classpath*. Une classe commune qui importerait Spring ne compilerait pas — et c'est voulu : un contrat RMI n'a pas besoin d'un conteneur.

2.4. Le serveur : vet-rmi-boot/vet-server/pom.xml

Listing 3 – Fichier : vet-rmi-boot/vet-server/pom.xml

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0">
3    <modelVersion>4.0.0</modelVersion>
4
5    <parent>
6      <groupId>fr.umontpellier.hai704i</groupId>
7      <artifactId>vet-rmi-boot</artifactId>
8      <version>1.0.0</version>
9    </parent>
10
11    <artifactId>vet-server</artifactId>      <!-- groupId et version herites -->
12
13    <dependencies>
14      <!-- les contrats partages -->
15      <dependency>
16        <groupId>fr.umontpellier.hai704i</groupId>
17        <artifactId>vet-common</artifactId>
18        <version>${project.version}</version>
19      </dependency>
20      <!-- le starter minimal : ni web, ni Tomcat, ni REST -->
21      <dependency>
22        <groupId>org.springframework.boot</groupId>
23        <artifactId>spring-boot-starter</artifactId>
24      </dependency>
25    </dependencies>
26
27    <build>
28      <plugins>
29        <!-- produit le JAR executable target/vet-server-1.0.0.jar -->
30        <plugin>
31          <groupId>org.springframework.boot</groupId>
32          <artifactId>spring-boot-maven-plugin</artifactId>
33        </plugin>
34      </plugins>
35    </build>
36  </project>
```

`${project.version}` reprend la version du projet (1.0.0), écrite une seule fois dans le POM racine. La version de `spring-boot-starter` est omise : elle vient du parent. `spring-boot-maven-plugin` produit un JAR exécutable qui embarque `vet-common` et Spring.

2.5. Le client : vet-rmi-boot/vet-client/pom.xml

Listing 4 – Fichier : vet-rmi-boot/vet-client/pom.xml

```
1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0">
3    <modelVersion>4.0.0</modelVersion>
4
5    <parent>
6      <groupId>fr.umontpellier.hai704i</groupId>
7      <artifactId>vet-rmi-boot</artifactId>
8      <version>1.0.0</version>
9    </parent>
10
11    <artifactId>vet-client</artifactId>      <!-- groupId et version herites -->
12
13    <dependencies>
14      <!-- les contrats partagés ; AUCUNE dépendance vers vet-server -->
15      <dependency>
16        <groupId>fr.umontpellier.hai704i</groupId>
17        <artifactId>vet-common</artifactId>
18        <version>${project.version}</version>
19      </dependency>
20      <!-- le starter minimal : ni web, ni Tomcat, ni REST -->
21      <dependency>
22        <groupId>org.springframework.boot</groupId>
23        <artifactId>spring-boot-starter</artifactId>
24      </dependency>
25    </dependencies>
26
27    <build>
28      <plugins>
29        <!-- produit le JAR exécutable target/vet-client-1.0.0.jar -->
30        <plugin>
31          <groupId>org.springframework.boot</groupId>
32          <artifactId>spring-boot-maven-plugin</artifactId>
33        </plugin>
34      </plugins>
35    </build>
36  </project>
```

Les dépendances ressemblent à celles du serveur, mais c'est un autre module : son propre `artifactId`, son propre JAR, et **aucune** dépendance vers `vet-server`. Une dépendance propre au client s'ajouterait ici, et seulement ici.

ET SPRING INITIALIZR ?

Le projet à construire est **un seul** projet Maven multi-modules, décrit par les quatre `pom.xml` ci-dessus. Si vous utilisez start.spring.io (ou l'assistant de votre IDE : Maven, Java, Jar, Java 17, aucune dépendance) pour obtenir un squelette de `vet-server` ou de `vet-client`, n'en gardez que les sources : le `pom.xml` engendré décrit un projet *autonome*, dont le parent est directement `spring-boot-starter-parent`. Remplacez-le par le `pom.xml` du module montré ci-dessus, qui hérite de `vet-rmi-boot`. Initializr ne propose plus la ligne 3.5 : la version 3.5.16 n'est écrite qu'une fois, dans le POM racine. `vet-common`, simple module Java, se crée à la main.

3. Côté serveur

Trois classes suffisent, plus un fichier de propriétés.

3.1. Le servent devient un bean

```
1 @Service
2 public class CabinetService extends UnicastRemoteObject implements ICabinet {
3
4     private final PatientRegistry registry;    // bean metier, sans RMI
5     private final String name;
6
7     // Injection PAR CONSTRUCTEUR. Depuis Spring 4.3, @Autowired est
8     // inutile quand la classe n'a qu'un seul constructeur.
9     public CabinetService(PatientRegistry registry,
10                          @Value("${vet.cabinet.name}") String name)
11         throws RemoteException {
12         super();                                // exportation RMI
13         this.registry = registry;
14         this.name = name;
15     }
16
17     @Override
18     public int countPatients() { return registry.count(); }
19     // ...
20 }
```

Ce qui vient de Spring : la création de l'instance, l'injection de `PatientRegistry`, la valeur lue dans les propriétés, le cycle de vie du bean. Ce qui reste du Java RMI : `extends UnicastRemoteObject`, `implements ICabinet`, `throws RemoteException`.

3.2. Le registre et la publication, dans une classe de configuration

```
1 @Configuration
2 public class RmiServerConfig {
3
4     @Value("${vet.rmi.port}")    private int port;
5     @Value("${vet.rmi.service-name}") private String serviceName;
6
7     // Le registre devient un bean : Spring en gere la duree de vie.
8     // destroyMethod="" car Registry n'a pas de methode close().
9     @Bean(destroyMethod = "")
10    public Registry rmiRegistry() throws Exception {
11        return LocateRegistry.createRegistry(port);
12    }
13
14    // Publication au demarrage : Spring injecte le registre et le servent,
15    // tous deux deja construits (donc le servent est deja exporte).
16    @Bean
17    public ApplicationRunner publishCabinet(Registry registry, ICabinet cabinet) {
18        return args -> registry.rebind(serviceName, cabinet);
19    }
20
21    // Arret propre : en vanilla, ce code devait etre ecrit a la main ;
22    // ici c'est le conteneur qui declenche la destruction du bean.
23    @Bean
24    public AutoCloseable rmiShutdown(Registry registry, ICabinet cabinet) {
25        return () -> {
```

```

26         registry.unbind(serviceName);
27         UnicastRemoteObject.unexportObject(cabinet, true);
28     };
29 }
30 }

```

3.3. Le point d'entrée et les propriétés

```

1 @SpringBootApplication
2 public class ServerApplication {
3     public static void main(String[] args) {
4         SpringApplication.run(ServerApplication.class, args);
5     }
6 }

```

```

1 # src/main/resources/application.properties (serveur)
2 spring.main.web-application-type=none
3 spring.main.keep-alive=true
4
5 vet.cabinet.name=Cabinet des Ecrins
6 vet.rmi.port=1099
7 vet.rmi.service-name=CabinetVeterinaire

```

LA PROPRIÉTÉ SPRING.MAIN.KEEP-ALIVE

Une application Spring Boot non web s'arrête dès que ses *runners* ont fini. Un serveur RMI n'a pas de *thread* non démon qui la maintienne en vie : sans cette propriété, votre serveur démarre, publie, puis s'arrête aussitôt. `spring.main.keep-alive=true` est disponible depuis Spring Boot 3.2.

4. Côté client

Le `lookup` n'a plus lieu au milieu du code métier : il est fait une fois dans une classe de configuration, et le stub est **injecté** là où il sert.

```

1 @Configuration
2 public class RmiClientConfig {
3
4     @Value("${vet.rmi.host}")         private String host;
5     @Value("${vet.rmi.port}")        private int port;
6     @Value("${vet.rmi.service-name}") private String serviceName;
7
8     @Bean(destroyMethod = "")
9     public Registry rmiRegistry() throws Exception {
10         return LocateRegistry.getRegistry(host, port);
11     }
12
13     @Bean
14     public ICabinet cabinet(Registry registry) throws Exception {
15         return (ICabinet) registry.lookup(serviceName); // RMI natif
16     }
17 }

```

```

1 @Component
2 public class ClientRunner implements ApplicationRunner {
3
4     private final ICabinet cabinet;
5
6     public ClientRunner(ICabinet cabinet) { // injection par constructeur
7         this.cabinet = cabinet;
8     }
9
10    @Override
11    public void run(ApplicationArguments args) throws Exception {
12        // Construire VOTRE logique client (etape A8) a partir du stub injecte,
13        // puis lancer VOTRE interface en ligne de commande, sans la reecrire.
14    }
15 }

```

Rien dans la CLI ni dans la logique client de l'étape A8 ne doit changer : Spring fournit le stub, votre code s'en sert comme avant. Si votre logique client est une classe ordinaire, elle peut aussi devenir un `@Bean` de `RmiClientConfig`, injecté dans le runner.

4.1. Le point d'entrée du client : fermer le contexte

```

1 @SpringBootApplication
2 public class ClientApplication {
3     public static void main(String[] args) {
4         ConfigurableApplicationContext contexte =
5             SpringApplication.run(ClientApplication.class, args);
6         System.exit(SpringApplication.exit(contexte));
7     }
8 }

```

UN CLIENT ABONNÉ AUX ALERTES NE S'ARRÊTE PAS TOUT SEUL

`SpringApplication.run` rend la main quand la CLI se termine, mais un observateur **exporté** garde un fil d'exécution RMI actif : sans rien faire, la JVM cliente reste en vie indéfiniment. `SpringApplication.exit` ferme le contexte, ce qui déclenche les méthodes de destruction des beans — l'occasion de désexporter l'observateur — puis `System.exit` termine le processus. Le client, contrairement au serveur, n'utilise pas `keep-alive`.

5. Construire et exécuter

```

$ cd vet-rmi-boot
$ mvn clean install
# ordre : vet-rmi-boot, vet-common, vet-server, vet-client -> BUILD SUCCESS

# terminal 1 : le serveur
$ java -jar vet-server/target/vet-server-1.0.0.jar

# terminal 2 : le client
$ java -jar vet-client/target/vet-client-1.0.0.jar

```

Depuis l'IDE, exécutez simplement les deux classes annotées `@SpringBootApplication`, dans cet ordre.

CE QUE VOUS DEVEZ OBSERVER

Dans la console du **serveur** :

```
INFO RmiServerConfig - registre RMI cree dans cette JVM sur le port 1099
INFO RmiServerConfig - service 'CabinetVeterinaire' publie
```

Dans celle du **client**, avant le menu de votre CLI :

```
INFO ClientRunner - stub injecte par Spring : jdk.proxy2.$Proxy31
(proxy dynamique : true)
```

Le stub reste un `java.lang.reflect.Proxy` : Spring l'a *injecté*, il ne l'a pas *fabriqué*. C'est toujours le runtime RMI qui le produit. Après « quitter », le processus client se termine.

6. La question de fond

CE QUE SPRING BOOT PREND EN CHARGE

création et assemblage des objets
injection des dépendances par constructeur
configuration externalisée (`application.properties`)
démarrage et arrêt ordonnés du contexte
journalisation, profils, tests

CE QUI RESTE DU JAVA RMI

`extends UnicastRemoteObject` : l'exportation
`LocateRegistry` : créer ou joindre le registre
`rebind / lookup` : publier et trouver
le stub, engendré par le runtime RMI
`RemoteException` : la défaillance partielle ne disparaît pas

CE QU'ON ATTEND DANS LE COMPTE-RENDU

Le point important est de distinguer deux choses que l'on confond souvent : Spring Boot a simplifié l'**assemblage** et le **cycle de vie** de votre application ; il n'a rien changé à la **communication distante**, qui reste exactement le Java RMI de la partie A — avec les mêmes contraintes, les mêmes exceptions et les mêmes questions de sémantique de passage.